COURSERA · PEER-GRADED ASSIGNMENT · WRITTEN REPORT

Exploring AI Tools in Software Development

Playing with AI Project & Peer Review

AI-assisted software development

PROJECT TITLE

Using ChatGPT to Assist in Developing a Student Task Manager Application

STUDENT AND SUBMISSION DETAILS

Student name	KRISHA KADAM
Roll number	A414
SAP ID	70022500747
Institution	[Institution name]
Course	Exploring AI Tools in Software Development
Assignment	Playing with AI Project & Peer Review (written report)
Platform	Coursera
Submission deadline	26 October 2026, 12:29 PM IST
Date prepared	[DD Month 2026]

STATUS OF EVIDENCE

Evidence note: observation, screenshot and pass/fail fields in this report are marked “pending verification” until the experiment has been executed.

Table of Contents

Executive Summary . . . 4

Section 1: Setup and Introduction . . . 4

1.1 Introduction to AI-assisted software development . . . 4
1.2 What ChatGPT is and its intended purpose . . . 4
1.3 Capabilities relevant to this experiment . . . 5
1.4 Why ChatGPT was selected for this experiment . . . 5
1.5 Objective and scope of the task manager project . . . 5
1.6 Technologies and development environment . . . 5
1.7 How conversational prompts guide software development . . . 5
1.8 Limitations of AI-generated code . . . 6
1.9 Human review, testing, privacy and responsible use . . . 6

Section 2: Experimentation and Documentation . . . 6

2.1 Project overview . . . 6
2.2 Requirements . . . 7
2.3 Tools and technologies . . . 7
2.4 Experimental procedure . . . 7
2.5 Example prompts . . . 8
2.6 Code example . . . 9
2.7 Challenges and limitations . . . 9
2.8 Experiment log . . . 10
2.9 Testing table . . . 11

Section 3: Effectiveness Analysis . . . 11

3.1 Approach to the analysis . . . 11
3.2 Potential strengths to look for . . . 13
3.3 Potential weaknesses to look for . . . 13
3.4 Plausible code versus verified code . . . 13
3.5 Questions specific to this experiment . . . 14

Section 4: Reflection and Future Implications . . . 14

Section 5: Conclusion . . . 15

References . . . 17

Appendix A: Example Prompts and Prompt Record Sheet 19

Prompt record sheet 19

Appendix B: Experiment Log 20

Appendix C: Test Cases and Verification Checklist 21

Appendix D: Screenshots and Evidence 22

Appendix E: Final Reflection Notes 23

Executive Summary

Software development is increasingly supported by artificial intelligence (AI) tools that can draft code, explain unfamiliar constructs and suggest fixes. This report documents an experiment for the Coursera course “Exploring AI Tools in Software Development” in which ChatGPT, a conversational AI system published by OpenAI, is used as an assistant across the activities of a small software project: building a browser-based student task manager with HTML, CSS and JavaScript.

The project was chosen because its scope is small enough for a student to read, run and test completely, yet it exercises a realistic range of development activities: writing requirements, generating an implementation, explaining generated code, debugging, designing test cases and producing documentation. The experiment follows a twelve-step procedure (Section 2.4) that runs from requirements through prompting, code generation, human review, execution, testing, targeted debugging, revision and comparison to a final evaluation against the requirements.

Effectiveness is assessed against ten criteria, including relevance of generated code to the requirements, speed of the first draft, readability, explanations, debugging support, test-case suggestions, edge-case handling, consistency, the human correction required and security risks. For each criterion, Table 7 states how it will be evaluated and what evidence is required, so that any judgement rests on recorded observations rather than on impressions.

Evidence status: the procedure, prompts, log and test suite supplied here are complete and reproducible, but every field reserved for an observed result is marked pending verification until the experiment has been executed and the material attached (Sections 2.8 and 2.9, Appendix D). Expectations about what the tool will return are therefore written as questions to investigate rather than as findings. The significance of the work lies in its method: it shows how a student can work with an AI assistant while human review, testing and accountability remain at the centre of the process.

How to read this report: procedures, prompts and test suites are complete as supplied; only records produced by the experiment may be reported as findings, and every field reserved for an observed result is marked pending verification until the run is complete.

Section 1: Setup and Introduction

1.1 Introduction to AI-assisted software development

A software project normally passes through a familiar sequence of activities: understanding what the user needs, designing a solution, writing code, running it, fixing defects, testing against the requirements and documenting it. AI-assisted software development means using a machine-learning system to take a share of those activities. Current tools hold a conversation, accept plain-language instructions and return code fragments, explanations, test ideas and documentation drafts within seconds. The defining characteristic of this workflow is that the AI produces a proposal rather than a finished product: its output must still be executed, checked and then accepted or rejected by a human developer.

1.2 What ChatGPT is and its intended purpose

ChatGPT is a conversational AI application published by OpenAI [1]. It is built on a large language model (LLM), a neural network trained on very large amounts of text so that it can predict plausible continuations of a given input [2]. A user types a message (a prompt), the model returns a reply, and earlier turns stay part of the context. ChatGPT is intended as a general-purpose assistant for answering questions, drafting and summarising text, analysis, brainstorming and programming help. Two limitations follow from that design: the model does not run or compile code, so it cannot observe whether its own suggestion works, and replies are generated probabilistically, so the same prompt can yield different answers.

1.3 Capabilities relevant to this experiment

The capabilities that matter here are: (i) code generation from a natural-language description of features; (ii) code explanation, where the tool describes what a function does and why; (iii) debugging assistance, where it proposes likely causes and candidate corrections for a reported symptom; (iv) brainstorming of alternative designs; (v) documentation drafting, such as comments or a README file; and (vi) test-case suggestions. Each is a capability to be tested in this experiment, not a guarantee of correctness.

1.4 Why ChatGPT was selected for this experiment

ChatGPT was selected because it is widely accessible to students, it covers every activity the experiment needs in a single interface, and its chat format keeps the whole interaction — prompt, response, follow-up, correction — visible and easy to record as evidence. Editor-based completion tools also assist, but their suggestions arrive silently and are harder to document step by step. The conversational form also allows deliberate prompt refinement, a skill this course aims to develop.

1.5 Objective and scope of the task manager project

The objective is to investigate how far ChatGPT can assist a student in producing a small but complete web application, and how much human review and testing is needed before it can be considered correct. The application is a single-page student task manager that adds tasks, displays them, marks them complete, deletes them, rejects empty input and refreshes the interface after each action. It is deliberately small so that every line can be read and the quality of the assistance can be judged rather than trusted. Servers, databases, accounts, deployment and frameworks are out of scope; task data is held in memory.

1.6 Technologies and development environment

Three technologies are enough. HTML (HyperText Markup Language) defines the page structure — the input field, the button and the list [4], [12]. CSS (Cascading Style Sheets) defines the presentation — spacing, colours and the visual treatment of completed tasks [5]. JavaScript defines the behaviour — validating input, changing the data and re-rendering the list [6]. The browser is both the execution and the test environment: opening the file runs the code and the developer console reports errors. Any modern browser and any text editor are sufficient; there is no build step.

1.7 How conversational prompts guide software development

A prompt is the instruction given to the tool. Here it acts as a lightweight specification: it names the goal, lists the features, states the constraints (no frameworks, beginner-readable code, three separate files) and describes the expected format of the answer. Vague prompts produce vague output, so prompts are refined across turns — first for a plan, then for code, then for an explanation, then for a targeted fix. The quality of the dialogue is therefore part of what is studied: a clearer prompt should require less correction.

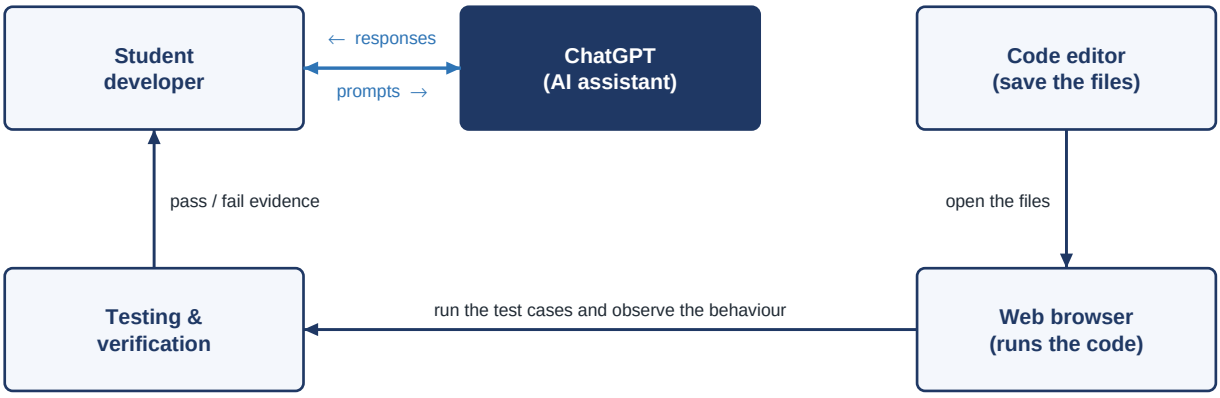


Figure 1. *Interaction between the developer, ChatGPT, the code editor, the browser and the testing step. Responses may contain code, explanations and suggested fixes, which the developer copies into the editor; only the browser executes code.*

1.8 Limitations of AI-generated code

Code that reads plausibly is not the same as code that has been verified [3]. Typical failure modes include syntax errors that prevent the script from loading, logic that quietly violates a requirement, missing handling of edge cases such as blank input, references to identifiers that do not exist in the project, and security weaknesses — for example inserting user-typed text with innerHTML, which can allow unintended markup to be interpreted as HTML [8]. The model cannot execute the code it writes, so none of these problems are detected by the tool itself; they surface only when a person runs the program.

1.9 Human review, testing, privacy and responsible use

Three responsibilities remain with the developer. First, review: AI output is read line by line against the requirements before it is used, and anything not understood is questioned. Second, testing: the program is executed in the browser and exercised against the test cases in Table 6, edge cases included, before any claim of correctness. Third, responsible use: no personal or confidential material is pasted into the chat, the tool's terms of service are respected [1], and AI assistance is declared as the course requires. Accountability for the code and the report stays with the student, not with the tool.

Section 2: Experimentation and Documentation

Project title: Using ChatGPT to Assist in Developing a Student Task Manager Application.

2.1 Project overview

The proposed application keeps a single list of tasks for the current session, letting the user add an item, see everything in one place, strike through what is finished and remove what is no longer needed. The intended user is a university student working on a laptop browser; the expected features are the six functional requirements listed in Table 1. The application is intentionally modest so that the experiment measures the quality of the AI assistance rather than the difficulty of the software.

2.2 Requirements

Requirements were written before any prompt was sent, so that the generated code could be judged against a fixed target rather than against whatever the tool happened to produce. Table 1 separates functional requirements (what the program must do) from basic quality requirements (how the program must behave and be written). Each requirement is written so that it can be checked by a single test case, which later supports the mapping between requirements and tests.

2.3 Tools and technologies

The experiment uses ChatGPT as the AI assistant, HTML and CSS for structure and presentation, JavaScript for behaviour, a plain text editor for saving the files and a modern browser as the combined execution and testing environment. Table 2 states the role of each tool and the evidence it produces. Keeping the toolchain this small also makes the experiment easy to reproduce.

2.4 Experimental procedure

The procedure below is written so that a second person could repeat the experiment and obtain comparable material. Each step states its purpose, the action to be taken, the learning outcome it should produce and the evidence to be collected. Steps 1 to 4 build the baseline; steps 5 to 8 establish what happens when the code is actually run; steps 9 to 11 cover targeted debugging and the comparison of versions; step 12 judges the final implementation against the requirements. The steps are iterative in practice: if testing reveals a defect, the developer returns to step 9 with a precise description of the observed behaviour rather than a guess.

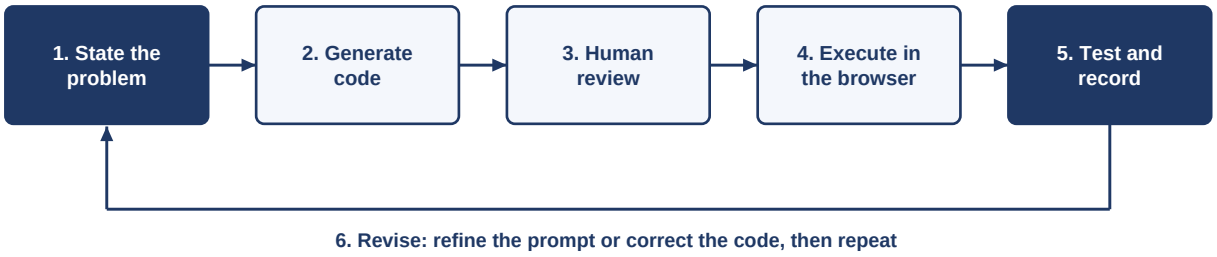


Figure 2. *Iteration loop used in the experiment: prompt, generated code, human review, execution in the browser, testing and revision.*

Table 3. Twelve-step experimental procedure: purpose, action, expected learning outcome and evidence to collect

No.	Purpose	Action to be taken	Expected learning outcome	Evidence to collect
1	Define the problem and the requirements	Write the functional and quality requirements in Table 1 and decide the scope boundary before opening the chat.	Requirements must be testable; a specification is what makes later judgement possible.	Dated requirements document (Table 1).
2	Prepare an initial prompt	Draft prompt P1: role, goal, six features, constraints (no frameworks, three files, beginner-readable code) and required output format.	Prompt structure controls output quality; ambiguity in, ambiguity out.	Exact prompt text, date and time.
3	Request an implementation plan	Send P2 and ask for the file structure, the in-memory data structure and the main functions before any code is accepted.	Reviewing a design is cheaper than reviewing a full listing.	Plan response, saved to the transcript file.
4	Generate an initial code draft	Send P3 (or ask for the code directly after the plan) and save the three files as version 1 without editing them yet.	Establishes the unmodified baseline against which all later changes are compared.	Version 1 files and the raw response.

No.	Purpose	Action to be taken	Expected learning outcome	Evidence to collect
5	Review the suggested code	Read version 1 line by line: missing functions, wrong element identifiers, logic that contradicts a requirement, unsafe rendering of user text.	Static review finds defects before execution and trains close reading.	Annotated review notes with line references.
6	Run the code in a suitable environment	Open index.html in a modern browser; observe the first screen and any console messages.	Confirms whether generated code executes at all in the real environment.	Screenshot of the running page; console output.
7	Test each required feature	Execute test cases TC-01 to TC-12 (Table 6) in order and fill the actual-result column immediately.	Expected behaviour fixed in advance cannot be adjusted to match the outcome.	Completed Table 6 with verdicts.
8	Record errors and unexpected behaviour	For each failure, write down the exact steps, the observed behaviour and the exact console message — the symptom, not a theory.	Precise symptoms are what makes a useful follow-up prompt possible.	Rows added to the experiment log (Table 5).
9	Ask ChatGPT for targeted debugging assistance	Send P5 containing the observed symptom, the console message and only the relevant code fragment.	Learning to describe a defect precisely; testing whether the tool diagnoses from evidence.	Prompt P5 and the returned suggestion.
10	Review and test any suggested correction	Apply the smallest accepted change as version 2, re-run the failed test case, then re-run the whole suite to check for regressions.	A fix is itself unverified until re-tested; regression testing prevents partial repairs.	Version 2 files; re-run of Table 6.
11	Compare the initial and revised versions	Place version 1 and version 2 side by side (diff or annotated notes) and list what changed and why.	Makes the contribution of each round explicit rather than assumed.	Comparison notes or a diff.
12	Evaluate against the requirements	Complete the observation column of Table 7 criterion by criterion and answer the questions in Section 3.5 from the records.	Converts raw material into an evidence-based judgement of effectiveness.	Filled Table 7 and written interpretation.

2.5 Example prompts

The prompts below are written examples of the kind of instruction used in the experiment; they are not transcripts of an actual conversation and must be replaced by, or compared with, the real prompts and responses recorded during the run (Appendix A). Each prompt is self-contained: it states the role and audience, the features required, the constraints to respect and the shape of the expected answer.

P1 · Generate the initial application

> You are helping a university student build a beginner-level web app. Write a single-page student task manager using only HTML, CSS and JavaScript (no frameworks and no external libraries). Features: add a task from a text input, show all tasks, mark a task as completed, delete a task, show a message and add nothing when the input is empty, and update the list immediately after every action. Keep the code readable for a beginner, use meaningful names, and give me three files: index.html, style.css and script.js.

P3 · Explain the generated code

> Explain the script.js code you just produced. For each function, describe what it does, which event triggers it, and how the list on the screen is updated. Assume I am new to JavaScript and point out anything you think a beginner is likely to misunderstand.

P4 · Identify potential bugs

> Review this task manager for bugs before I run it. Focus on: empty or whitespace-only input, deleting the wrong item, items disappearing after an update, duplicate identifiers, and any place where user-typed text could be treated as HTML. List each issue, the line it occurs on, and why it would fail.

P6 · Design test cases

> List test cases for this task manager covering adding tasks, displaying them, completing them, deleting them, empty input, whitespace-only input, long text and repeating each action several times. For every test case give the steps, the expected result, and what would count as a failure.

P7 · Improve readability without changing behaviour

> Improve the readability of this function without changing its behaviour: rename unclear variables, split the function into smaller ones, and add a brief comment only where the logic is not obvious. Then tell me exactly what you changed and why.

P8 · Produce documentation

> Write a short README for this project: what the application does, which files it contains, how to run it locally, how to test it manually, and the known limitations of the current version. Use plain language and do not claim features that are not in the code.

2.6 Code example

The following JavaScript fragment is an illustrative example of the code the experiment is expected to produce for the first functional requirement. It is kept short so that it can be discussed line by line; it is not evidence of an experimental result.

```
// Adds one task after checking the input. (Illustrative example.)
function addTask() {
  const input = document.getElementById("taskInput");
  const text = input.value.trim();

  if (text === "") {
    showError("Please type a task before adding.");  // empty-input rule
    input.focus();
    return;                                          // change nothing
  }

  const task = { id: Date.now(), text: text, done: false };
  tasks.push(task);        // update the data first ...
  renderTasks();           // ... then refresh the whole list
  input.value = "";        // clear the field for the next task
  input.focus();
}
```

Listing 1. *Illustrative addTask() function for the task manager. The snippet is shown for review and discussion; it is not an experimental result.*

The function reads the value of the input field and trims surrounding whitespace. If nothing remains, it reports the problem and returns without changing the data, which covers the empty-input requirement. Otherwise it creates a task object, appends it to the tasks array, calls renderTasks() to redraw the list and clears the field for the next item. The example concentrates three requirements in one place — input validation, state change and interface refresh — so it can be checked by reading, by running the corresponding test cases and by asking the tool to explain its choices.

2.7 Challenges and limitations

Table 4 lists the challenges that are reasonable to anticipate when an AI tool is used this way. These are potential challenges to investigate: they are not problems that have been observed in this experiment. Anything actually encountered during the run is recorded verbatim in the experiment log (Table 5) with its verification status, so that the two categories are never confused.

Table 4. Challenges to investigate (potential, not yet observed)

Challenge	Why it may occur	How it would be recognised	Status
Requirement misunderstood	The prompt is ambiguous or the tool infers a different scope.	The code does something adjacent to what was asked; FR mapping shows a gap.	Potential
Syntax error, script does not load	Generated code is never executed by the tool before it is returned [3].	Blank page or console message naming a parse error.	Potential
Undefined function or wrong element id	Fragments produced separately do not line up.	Console reports “is not defined”, or a click produces no effect.	Potential
Edge case not handled	Blank input, very long text or rapid clicks were not named in the prompt.	TC-06, TC-07 or TC-08 fails during step 7.	Potential
Suggested correction does not work	The diagnosis is based on an incomplete description of the symptom.	The failed test still fails, or a new test fails after the change.	Potential
Inconsistent answers between runs	Replies are generated probabilistically, not retrieved from a fixed source.	The same prompt in a fresh chat returns a different structure.	Potential
Unsafe rendering of user text	Generated code may insert typed text as HTML rather than as text [8].	Markup typed as a task title is rendered by the browser.	Potential
Verbose or hard-to-follow explanation	Conversational format favours completeness over focus.	The actionable point is hard to locate in a long answer.	Potential

2.8 Experiment log

Table 5 is the running record of the experiment. It is filled in during the run, not afterwards, so that observations are not reconstructed from memory. The “ChatGPT assistance” column records what was asked and returned; the “actual observation” column records what really happened on screen or in the console; the verification status column shows whether the step has been checked, is still pending, or could not be reproduced.

Table 5. Experiment log — assistance column states what is planned at each step; observations and status are completed during the run

Step	Action performed	ChatGPT assistance	Actual observation	Verification status
1	Write the requirements (Table 1).	Optional: ask the tool to critique the list for gaps.	[Pending — record during the run]	Pending
2	Write the initial prompt P1.	None yet; the prompt is authored by hand.	[Pending — record during the run]	Pending
3	Send P2 and review the returned plan.	Plan (files, data structure, functions) returned by ChatGPT.	[Pending — record during the run]	Pending
4	Request code and save it unchanged as version 1.	Three-file draft returned by ChatGPT.	[Pending — record during the run]	Pending
5	Read version 1 line by line and annotate.	P3 used to explain any unclear part of the code.	[Pending — record during the run]	Pending
6	Open index.html in the browser and observe.	None; execution is performed by the browser.	[Pending — record during the run]	Pending
7	Run test cases TC-01 to TC-12 (Table 6).	None; the observed behaviour is the ground truth.	[Pending — record during the run]	Pending
8	Record every failure verbatim (symptom and console message).	None; recording must stay independent of the tool.	[Pending — record during the run]	Pending
9	Send P5 with the symptom, console message and relevant code.	Diagnosis and proposed patch returned by ChatGPT.	[Pending — record during the run]	Pending
10	Apply the accepted change as version 2 and re-test everything.	P3-style follow-up: explain why the change fixes the symptom.	[Pending — record during the run]	Pending
11	Compare version 1 with version 2.	Optional: ask the tool to summarise the difference.	[Pending — record during the run]	Pending

Step	Action performed	ChatGPT assistance	Actual observation	Verification status
12	Complete the observation column of Table 7 and answer Section 3.5.	None; the final judgement must be the student's own.	[Pending — record during the run]	Pending

2.9 Testing table

Table 6 defines the test cases used to evaluate the implementation. They cover each functional requirement, the boundaries around empty and whitespace-only input, repeated actions and the end state after everything has been deleted. Expected behaviour is stated in advance so that it cannot be adjusted to match the observed result. Actual results and verdicts remain pending until the tests have been executed in the browser and the evidence attached in Appendix D.

Table 6. Test cases: expected behaviour is fixed in advance; actual results and verdicts are filled after execution

ID	Test and steps	Expected behaviour	Actual result	Verdict
TC-01	Type “Submit lab report” and press Add (FR-1).	The task appears once in the list and the input field is cleared.	[Pending]	Pending
TC-02	Add three different tasks one after another (FR-2).	All three tasks are visible, in the order they were added.	[Pending]	Pending
TC-03	Activate the complete control on the first task (FR-3).	The task is shown as completed and remains in the list.	[Pending]	Pending
TC-04	Activate the complete control on the same task again (FR-3).	The task returns to the not-completed state.	[Pending]	Pending
TC-05	Delete the middle task of three (FR-4).	Only that task disappears; the other two are unchanged.	[Pending]	Pending
TC-06	Press Add with the input field empty (FR-5).	Nothing is added and a visible message explains the problem.	[Pending]	Pending
TC-07	Type only spaces and press Add (FR-5).	Nothing is added; the same message is shown.	[Pending]	Pending
TC-08	Add a task containing about 300 characters.	The task is displayed and the layout is not broken.	[Pending]	Pending
TC-09	Add five tasks in quick succession (FR-6).	Exactly five items are listed; no duplicates or missing items.	[Pending]	Pending
TC-10	Toggle and delete several times in a row (FR-6, QR-4).	The list always matches the last action; no console errors appear.	[Pending]	Pending
TC-11	Delete every task until the list is empty.	An empty list or an empty-state message is shown; no errors occur.	[Pending]	Pending
TC-12	Refresh the page after adding tasks.	Behaviour matches the declared scope (in-memory: list is cleared).	[Pending]	Pending

Section 3: Effectiveness Analysis

3.1 Approach to the analysis

The analysis evaluates how effectively ChatGPT supported the development of the task manager. Ten criteria were fixed before the experiment so that the assessment could not be shaped after the fact (Figure 4). For every criterion, Table 7 states the evaluation method, the evidence required, a placeholder for the actual observation and the interpretation rule — what different outcomes would mean. Until the experiment has been run the observation column reads “pending”. Writing the interpretation rules in advance is what allows a later reader to distinguish a genuine result from a story constructed to fit the outcome.

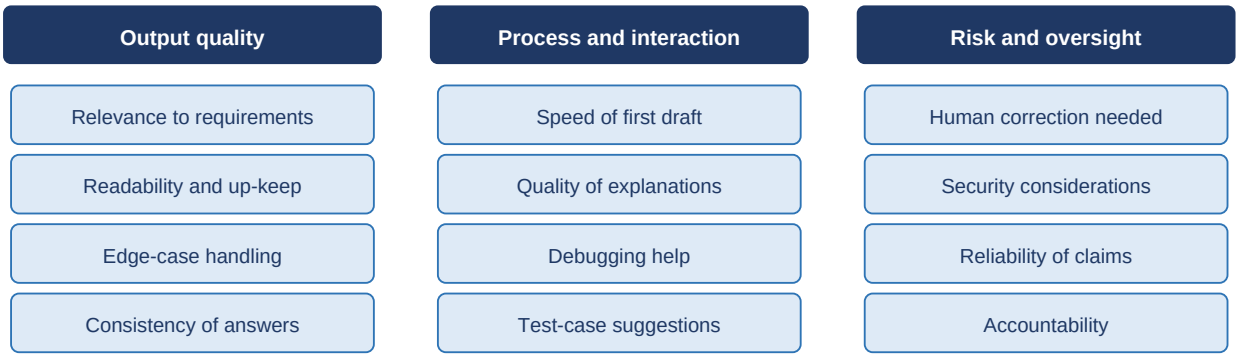


Figure 4. *The ten evaluation criteria grouped into output quality, process and interaction, and risk and oversight.*

Table 7. Evaluation criteria: method, required evidence, observation placeholder and interpretation rule

Criterion	Evaluation method	Evidence required	Actual observation	Interpretation if/when observed
Relevance of generated code to requirements	Map each requirement FR-1 to FR-6 to the code lines that implement it.	Annotated requirement-to-code mapping.	[Pending]	If every FR maps to working code, relevance is high; gaps or dead features indicate a prompt or comprehension problem.
Speed of creating an initial draft	Record the timestamp of the first prompt and of the first complete three-file response.	Conversation timestamps; note of any retries.	[Pending]	Compare elapsed times across attempts; no speed claim is made without timestamps.
Readability and maintainability	Independent read-through: unclear names, duplicated logic, length of functions.	Annotated version 1 listing; review notes.	[Pending]	Few unclear or duplicated elements suggest readable output; extensive rewriting suggests the opposite.
Helpfulness of explanations	Check whether each answer addresses the question asked and matches the code when read line by line.	Question-and-answer pairs checked against the listing.	[Pending]	Explanations that match the code support learning; contradictions must be corrected by the reader.
Assistance with debugging	Apply the suggested correction to a failing test case, then re-run that case and the full suite.	Before-and-after results from Table 6.	[Pending]	A fix that makes the failed case pass without breaking others is effective assistance.
Quality of test-case suggestions	Compare the tool's test list with Table 6: cases found, cases missed, cases irrelevant here.	Two lists side by side.	[Pending]	Useful additions raise confidence; missing edge cases show where human test design is still needed.
Handling of edge cases	Execute TC-06, TC-07, TC-08 and TC-11 on the unmodified first draft.	Verdicts for those test cases.	[Pending]	Passing before correction shows edge cases were anticipated; failing shows they must be demanded in the prompt.
Consistency of responses	Repeat the same prompt in a fresh conversation and compare the two answers.	Two saved transcripts of the same prompt.	[Pending]	Similar structures indicate consistency; different designs must be reconciled by the developer.
Amount of human correction required	List every edit made between version 1 and version 2 and classify it (defect, requirement, style).	Version comparison; count and classification of edits.	[Pending]	Many defect-class edits mean heavy dependence on review; mostly style edits mean the draft was close.
Security and reliability risks	Review how user-typed text is inserted into the page and how input is validated; compare with [8] and [9].	Security review notes; console error log.	[Pending]	Unsafe patterns found must be corrected before the result is called usable.

3.2 Potential strengths to look for

If the tool performs as its design suggests, four strengths should appear in the record. The first is speed: a complete first draft of three files can arrive in one response, so the developer reaches a runnable program sooner, and the time from prompt to draft can be measured from the conversation timestamps. The second is accessible explanation: asked to describe an unfamiliar construct such as a DOM event, the tool can usually explain it at the reader's level, which supports learning while working. The third is alternatives: a follow-up prompt can produce a different data structure or rendering approach, giving a choice that would otherwise require reading several tutorials. The fourth is the unglamorous work — test-case lists, comments and README text — produced quickly, which raises the chance that a beginner will test and document at all. Follow-up prompts also allow interactive refinement, narrowing the gap between what was asked for and what was returned.

3.3 Potential weaknesses to look for

The corresponding risks are equally concrete. Generated code may be incorrect or incomplete: it may parse cleanly and still violate a requirement, or call a function that was never defined. Requirements can be misread when the prompt is ambiguous — asking for a “list of tasks” leaves open whether items persist after a refresh. Edge cases are the first casualty: whitespace-only input, very long text, rapid repeated clicks and deleting the final item are often absent from a first draft unless named in the prompt. Responses are not fully consistent, so the same question in a fresh conversation may yield a different structure. Security weaknesses can enter quietly, for example rendering user-typed text with innerHTML instead of textContent [8]. Above all, none of these problems are visible until a person runs and tests the program, so the output is only as good as the verification that follows.

3.4 Plausible code versus verified code

The distinction at the centre of this analysis is between code that looks correct and code that has been shown to be correct in the environment where it runs. A generated function can be well formatted, sensibly named and logically convincing and still fail for a reason that only appears at runtime: a wrong element id, an event bound before the element exists, or an assumption about input that the browser does not enforce [3]. Verification means executing the program, observing each required behaviour, exercising the edge cases and recording the result — exactly the activity in steps 6 to 8 of the procedure. In this report, only statements supported by such a record may be called findings; everything else is a hypothesis, and is labelled as such.

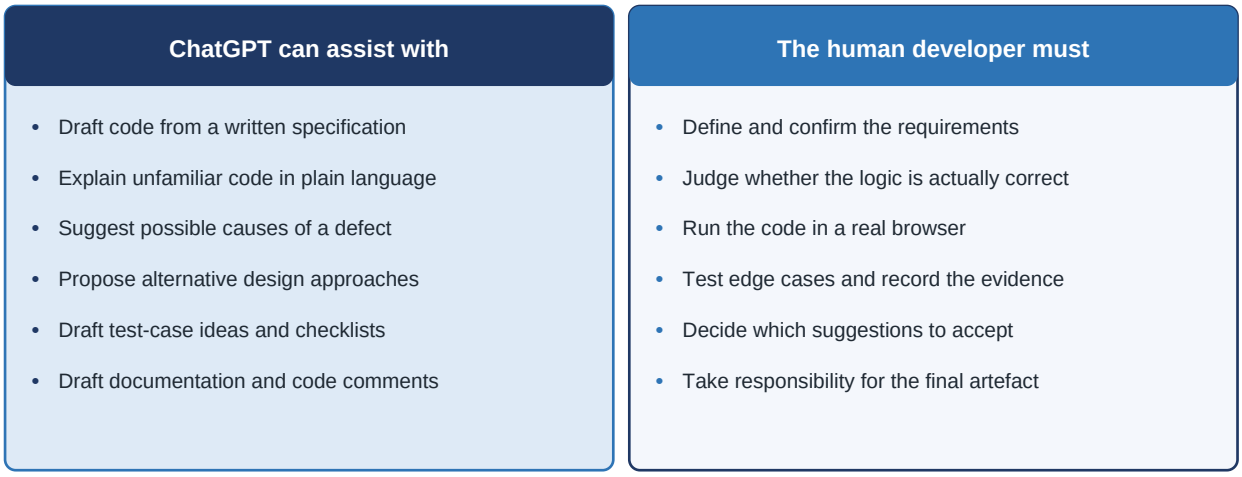


Figure 3. *Division of labour: the tool proposes, the developer reviews, runs, tests and remains accountable for the result.*

3.5 Questions specific to this experiment

The following questions are what the completed experiment is expected to answer, phrased as questions because the evidence is not yet in place: Does the first draft satisfy all six functional requirements without edits? How many separate corrections are needed before every test case in Table 6 passes? Does the tool suggest test cases the student had not thought of, and are they relevant here? Does its explanation of its own code match what the code does line by line? How does the response change if the same prompt is repeated in a new conversation? Which of the ten criteria in Table 7 needs little human correction, and which requires substantial rework?

Section 4: Reflection and Future Implications

This section is a draft for you to personalise. Bracketed prompts such as "[add your observation]" are placeholders: replace them with your own experience once the experiment has been run, and delete any statement that does not match what actually happened.

1. What can a student learn by using ChatGPT during software development?

Used carefully, the tool behaves like a patient tutor that never tires of follow-up questions. A student can learn how a problem is decomposed into files and functions, why one data structure is chosen over another, and what an unfamiliar line does — learning that normally requires reading several documentation pages. The condition is active reading: the explanation must be checked against the code instead of accepted. [Add the specific concept you understood better after this experiment.]

2. How can AI tools be incorporated into a normal programming workflow?

The most defensible place for AI is at the boundaries of the work: clarifying requirements, drafting a plan, producing a first skeleton, explaining unfamiliar code, proposing hypotheses when something breaks, generating a test checklist and drafting documentation. A workable routine is plan ☐ draft ☐ review ☐ run ☐ test ☐ revise, with the tool consulted at the plan, draft, explanation and debugging stages while the human keeps review and verification. Keeping AI-assisted changes in version control makes the tool's contribution visible and reversible. [Note where the assistance was most and least useful in your own workflow.]

3. Which development tasks are well suited to AI assistance?

Tasks with quick, unambiguous feedback and low stakes suit it best: boilerplate structure, routine CRUD-style functions, rewriting a snippet for clarity, drafting comments and README text, enumerating test scenarios and explaining error messages. In this project that includes the HTML skeleton, the list rendering, the documentation and the initial test list.

4. Which tasks require particularly careful human judgment?

Decisions that are hard to reverse or hard to detect: whether the requirements themselves are correct, whether the logic really implements them, how input is handled safely, what happens at the boundaries, whether a fix has broken something else, and whether the software is ready to be used or submitted. Errors here can pass unnoticed, because the tool gives no warning that it is uncertain.

5. Why must AI-generated code be reviewed and tested?

Because the generator never executes the program: it produces text that resembles working code, and resemblance is not evidence [3]. Review catches misunderstandings of the requirements; testing catches everything that only appears at runtime — wrong identifiers, unhandled input, events bound to elements that do not exist. Testing also turns an opinion about quality into a record that a reviewer or a future maintainer can check.

6. What are the risks of relying too heavily on AI?

Skill erosion is the quiet risk: if reading, writing and debugging are always outsourced, the ability to judge the output declines precisely when it is most needed. Other risks are silent requirement drift, duplicated or insecure patterns spreading across a project, over-trust in an authoritative tone, and unclear accountability when generated code fails. Practical risks also remain: material pasted into a chat may be retained by the service [1], and the provenance of generated fragments may be unclear.

7. How could AI influence software development in the future?

The likely direction is deeper integration into the development environment: tools that propose changes across several files, run tests themselves, explain failures and keep documentation in step with code. That would raise the baseline speed of routine work and shift attention towards specification, review and verification. It would also raise the cost of being unable to read code, since the volume of code one person oversees would grow.

8. What are the possible effects on learning, productivity, collaboration and developer responsibilities?

For learning, the effect depends on use: as a Socratic assistant it can accelerate understanding, as an answer machine it can short-circuit it. Drafts and documentation are clearly faster; written prompts and transcripts can make design reasoning easier to share and review. Accountability increases rather than decreases — submitting generated code means asserting that it works, a claim only the developer can support with tests. [Add how your own practice changed during this experiment.]

9. How can developers use AI responsibly?

By declaring its use where disclosure is required, by never entering confidential data into a chat, by reviewing output as one would review a colleague's pull request, by testing before claiming correctness, by respecting the terms of service and licences of the tool [1], and by staying accountable for what is delivered. Responsible use also means correcting the tool when it is wrong rather than keeping a flawed suggestion.

10. What would be improved in a future version of this experiment?

Three improvements are already identifiable before the run: repeat each prompt in a fresh conversation to measure consistency, record timestamps for every prompt so that speed claims rest on data, and add a small automated check so that verification is not purely manual. Extending the application with browser storage would allow persistence and refresh behaviour to be tested rather than assumed. [Add the improvements you identify after completing the run.]

Section 5: Conclusion

Provisional conclusion — update after the experiment has been executed and Sections 2.8, 2.9, 3 and 4 have been completed.

This report set out to examine how a conversational AI tool can assist a student through the full path of a small software project: a browser-based task manager built with HTML, CSS and JavaScript. The approach was to fix the requirements first, define a twelve-step procedure covering prompting, code generation, human review, execution, testing, debugging, revision and comparison, and to pre-register ten evaluation criteria together with the evidence each one requires. The material produced so far — requirements, prompts, an illustrative code example, the experiment log, the test suite and the verification checklist — is enough for another student to repeat the work.

The main limitation is equally clear: the observation fields are still empty. No conclusion about the tool's effectiveness can honestly be drawn until the program has been executed, the tests run and the transcripts and screenshots attached. This conclusion is therefore provisional. What can already be stated is the standard of proof the report applies: code is treated as a proposal until it has been reviewed line by line and observed

working in the browser, expected behaviour is never reported as an actual result, and every claim must be traceable to a record in Table 5, Table 6 or Table 7.

Three updates are required after testing. First, the observation columns of Tables 5, 6 and 7 must be completed and the status changed from pending to checked, with screenshots added in Appendix D. Second, the questions in Section 3.5 must be answered from that evidence, including the number of corrections needed before all test cases passed. Third, Sections 4 and 5 must be rewritten in the student's own words to reflect what happened, and the provisional label removed only if the experiment is complete. Only then may the report state whether the assistance was effective for this task, and on what evidence.

References

[1] OpenAI, “ChatGPT,” OpenAI, San Francisco, CA, 2023. [Online]. Available: https://openai.com/chatgpt/

[2] OpenAI et al., “GPT-4 technical report,” arXiv:2303.08774, 2023. [Online]. Available: https://arxiv.org/abs/2303.08774

[3] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, and A. Ray et al., “Evaluating large language models trained on code,” arXiv:2107.03374, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[4] WHATWG, “HTML living standard,” Web Hypertext Application Technology Working Group, 2025. [Online]. Available: https://html.spec.whatwg.org/

[5] MDN Web Docs, “CSS: Cascading Style Sheets,” Mozilla, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS

[6] MDN Web Docs, “JavaScript guide,” Mozilla, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

[7] W3C, “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, Oct. 2023. [Online]. Available: https://www.w3.org/TR/WCAG22/

[8] OWASP, “OWASP Top Ten,” Open Worldwide Application Security Project, 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

[9] NIST, “Secure Software Development Framework (SSDF) Version 1.1,” NIST Special Publication 800-218, Gaithersburg, MD, Feb. 2022. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/218/final

[10] ISO/IEC/IEEE, “Systems and software engineering — Software testing — Part 1: General concepts,” ISO/IEC/IEEE 29119-1:2022, Jan. 2022. [Online]. Available: https://www.iso.org/standard/81291.html

[11] IEEE, “IEEE Standard for Software Quality Assurance Processes,” IEEE Std 730-2014, Jun. 2014, doi: 10.1109/IEEESTD.2014.6835311.

[12] MDN Web Docs, “HTML: HyperText Markup Language,” Mozilla, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML

Rubric Alignment

The matrix below shows which sections of this written report address each marked criterion of the assignment. The video and the peer-review activity are separate deliverables and are not part of this document; the report does not claim to satisfy them.

Table 8. Rubric-alignment matrix for the written report (the assignment totals 100 marks across written report, video and peer review)

Criterion	Marks	Where it is addressed in this report	Deliverable
Setup: Completeness and clarity	5	Section 1.1–1.5; Figure 1	Report
Setup: Quality of tool description	5	Sections 1.2–1.4, 1.8–1.9; references [1]–[3]	Report
Experimentation: Thoroughness	10	Sections 2.1–2.4; Tables 2–3; Figure 2	Report
Experimentation: Documentation	10	Sections 2.5–2.6, 2.8–2.9; Tables 5–6; Appendices A–C	Report
Experimentation: Creativity	10	Prompts P4–P8; Sections 2.7, 3.5; edge-case tests TC-07–TC-12	Report
Analysis: Quality of analysis	15	Section 3; Table 7; Figures 3–4	Report
Reflection: Insightfulness	15	Section 4 (personalise); Section 5; Appendix E	Report
Presentation: Clarity and organization	10	Whole document: cover, contents, headings, tables, captions, references	Report
Presentation: Video effectiveness	10	—	Separate video deliverable
Peer review: Quality of feedback	10	—	Separate peer-review activity

Appendix A: Example Prompts and Prompt Record Sheet

Prompts P1, P3, P4, P6, P7 and P8 are printed in Section 2.5. The two further prompts used for planning and for targeted debugging are given below, followed by a blank record sheet in which the prompts actually used during the experiment, their date and time and the location of the saved response must be entered. Replace these examples with the genuine transcript material before submission.

P2 · Request an implementation plan first

> Before writing any code, describe the file structure, the data structure you would use to store the tasks in memory, and the main functions you plan to create. Keep it short and explain each choice in one sentence.

P5 · Targeted debugging with an observed symptom

> When I press Add with an empty input, here is what actually happens: [describe the observed behaviour]. The exact browser console message is: [paste error]. The relevant function is: [paste code]. Explain the most likely cause and give the smallest change that fixes it, without rewriting the rest of the file.

Prompt record sheet

Table 9. Prompt record sheet — paste the prompts actually used, with the date, time and where the response was saved

Prompt ID	Date and time	Actual prompt text used	Response saved at (file / link)
P1			
P2			
P3			
P4			
P5			
P6			
P7			
P8			

Appendix B: Experiment Log

Table 5 in Section 2.8 is the structured log keyed to the twelve steps. This appendix provides space for additional entries made during the run, including the time of each observation and a reference to the evidence (screenshot name, transcript file or console note) that supports it.

Table 10. Extended experiment log for further entries (continuation of Table 5)

Date and time	Step	Observation (verbatim)	Evidence reference

Appendix C: Test Cases and Verification Checklist

The test cases themselves are defined in Table 6 (TC-01 to TC-12). The checklist below is used to confirm that the evidence for each part of the experiment exists before the report is submitted. Each item must be ticked only when the corresponding material is present.

Verification checklist

- [ ] Requirements table (Table 1) agreed before prompting.
- [ ] Prompt P1 (or its real replacement) copied into Appendix A with date and time.
- [ ] Initial code draft saved as version 1 in a separate folder.
- [ ] Browser execution of version 1 recorded (screenshot or console note).
- [ ] All test cases in Table 6 executed, with actual results filled in.
- [ ] Any defect found described verbatim in Table 5 (symptom, not theory).
- [ ] Debugging prompt and response pasted into Appendix A.
- [ ] Version 2 saved after applying the accepted correction.
- [ ] Version 1 and version 2 compared (diff or annotated notes).
- [ ] Observation column of Table 7 completed for every criterion.
- [ ] Screenshots attached in Appendix D and referenced in the text.
- [ ] Section 4 and Section 5 updated from “pending” to the final conclusion.

Appendix D: Screenshots and Evidence

Insert genuine screenshots only. Useful captures are: the ChatGPT conversation showing each prompt and response; the browser window with the application running; the developer console before and after a fix; and the filled test table. Crop the images so that they fit the page width, and add the date and the step number to each caption.

D.1 ChatGPT conversation: initial prompt and first response	*Paste screenshot here*
D.2 The task manager running in the browser (version 1)	*Paste screenshot here*
D.3 Browser console showing any errors observed while testing	*Paste screenshot here*
D.4 The corrected version 2 after the debugging round	*Paste screenshot here*
D.5 Completed test table or a pass/fail summary	*Paste screenshot here*

Appendix E: Final Reflection Notes

Use this space for your own reflection after the experiment has been run. The paragraphs in Section 4 are drafts: replace bracketed placeholders with your own observations, delete anything you disagree with, and keep the balanced conclusion that AI assistance does not remove the need for programming knowledge, testing or accountability.

Guiding questions for your own notes

- What surprised you about the first draft returned by the tool?
- Which requirement needed the most human correction, and why?
- Which test case failed for a reason you did not expect?
- What would you prompt differently if you repeated the experiment?
- What did you understand better after the experiment than before it?

End of report.